

Randomized algorithm for MIN-CUT in graphs

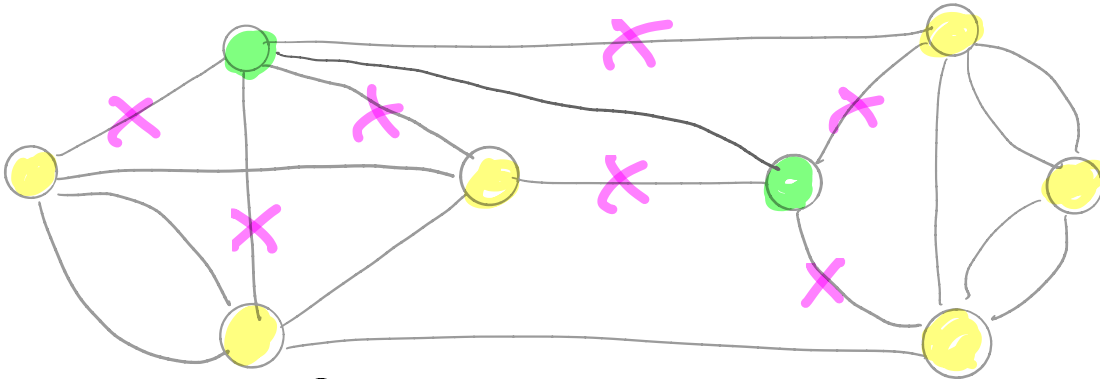
Graph $G = (V, E)$ (multiple edges)

$n = |V| = \# \text{ nodes}$

$m = |E| = \# \text{ edges}$ (multiplicities of the same edges)

► $C = (V_1, V_2)$ is a **CUT** if $V_1 \cap V_2 = \emptyset$ and $V_1 \cup V_2 = V$

CUTSET $E(V_1, V_2) = \{ uv \in E : u \in V_1, v \in V_2 \text{ or } u \in V_2, v \in V_1 \} \subseteq E$

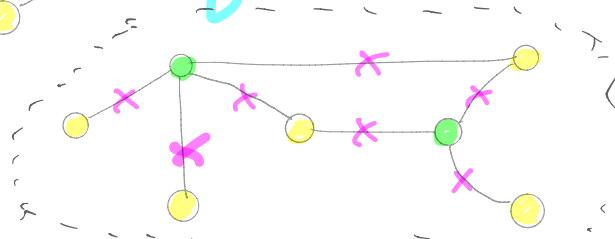
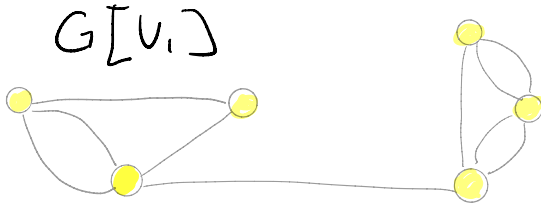
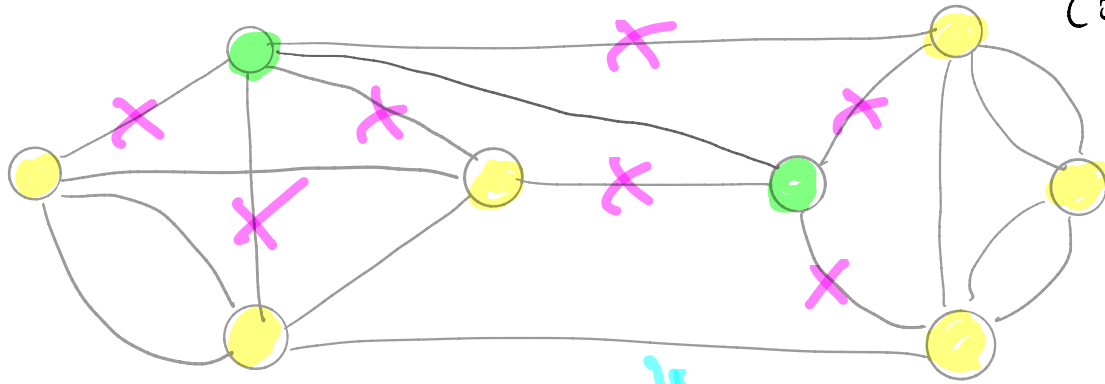


$|E(V_1, V_2)| = 7$

$n = 8, m = 18$

2-coloring of the nodes $\equiv$ CUT

A cut divides G into 3 parts: applications e.g. recursion on graphs (divide-and-conquer)



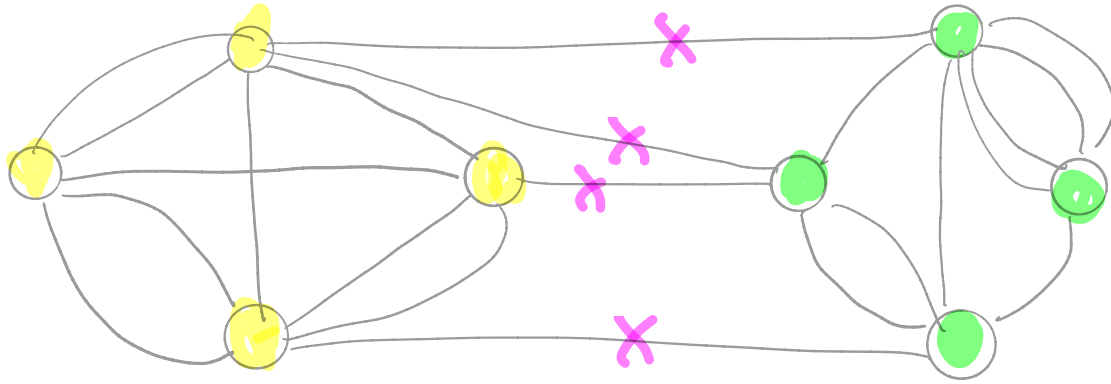
subgraph induced by the cutset
 $E(V_1, V_2) = E(C)$

& we want its size

MIN
poly-time

MAX
NP-hard

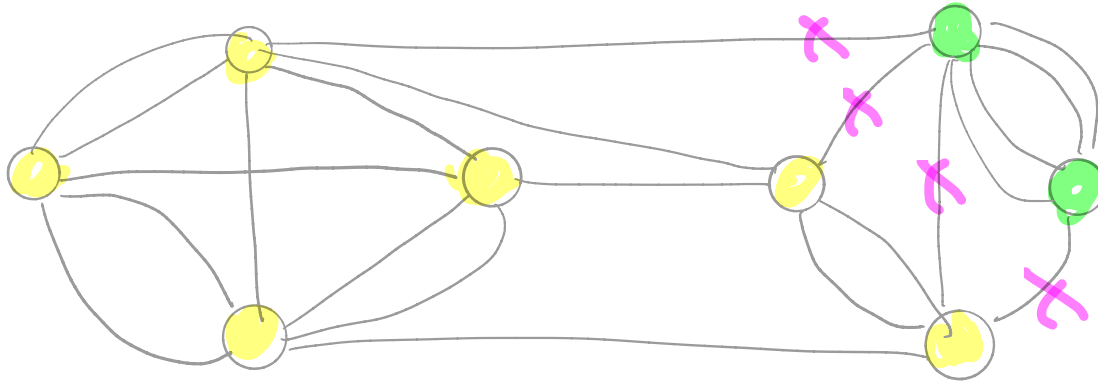
▶ $C = (V_1, V_2)$ is a MIN-CUT if $|E(C)| \leq |E(C')|$ for any cut C'



1 MIN-CUT is not unique

2 $|E(C)| \leq \deg(v)$
 $\forall v \in V$

(as otherwise we could choose a smaller cut-set)





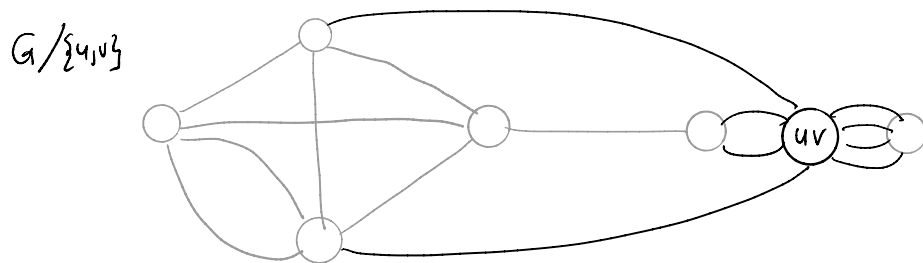
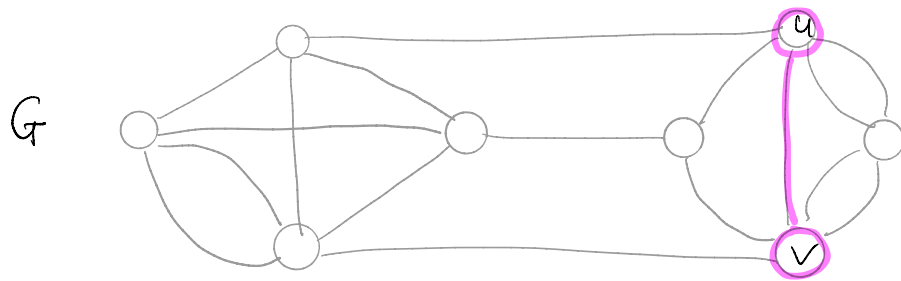
EDGE CONTRACTION

$uv \in E$ edge

$$G/uv = (V', E') \text{ where } V' = V - \{u, v\} \cup \{\textcircled{uv}\}$$

$$E' = E - \{uv\} \cup \{\textcircled{uv}z : uz \in E \text{ or } vz \in E\}$$

they can be
multiple edges
between u and v



INTUITION: We are conceptually assigning the same color to u and v in the final cut, so we can drop all the multi-edges uv

Each edge contraction decreases # nodes by 1

RELATION BETWEEN EDGE CONTRACTIONS AND CUTS

G connected

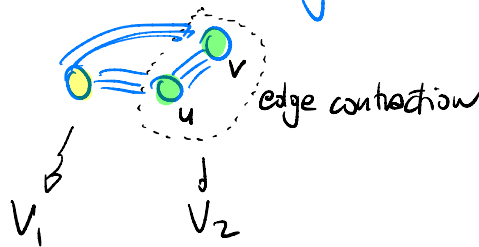
② Any sequence of edge contractions leaving just 2 nodes gives a cut V_1, V_2

By induction on the number n of nodes

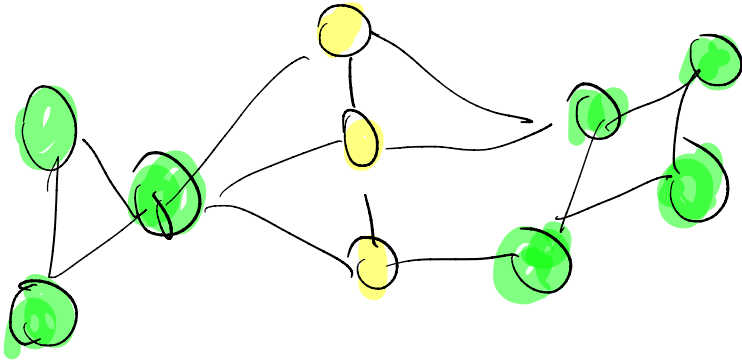
- $n = 2$: V_1 = one node V_2 = other node



- $n > 2$: take the last edge contraction, done on 3 nodes



⑥ Given a cut $C=(V_1, V_2)$, it does not necessarily exist a sequence of edge contractions that lead to just 2 nodes



no
sequence
of edge
contractions



here we cannot apply
any edge contraction
that keeps this 2-colouring
(clearly, by ②, we can
obtain another 2-colouring)

ALGORITHM GUESSMINCUT (G):

while ^{number of current nodes} $|V(G)| > 2$ do:

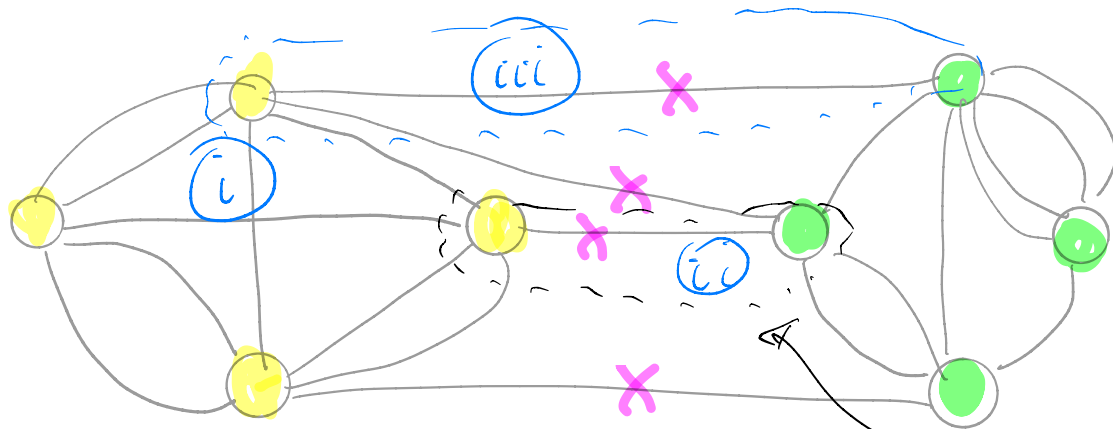
1) choose uniformly and randomly an edge $uv \in E$ (how?)

2) $G \leftarrow G/uv$

return $|E(G)|$

<sup>number of multiple edges
when only 2 nodes remain</sup>

Q: when does the above algorithm fail?

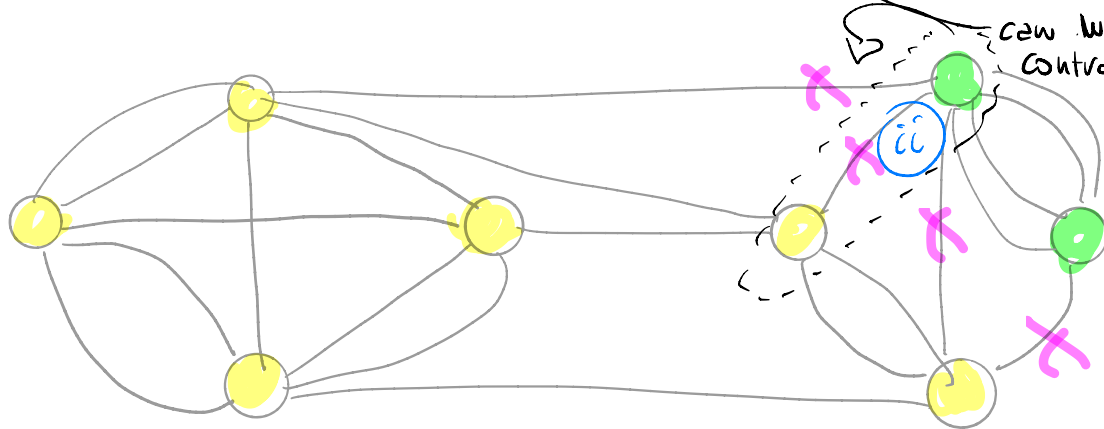


i

if uv is not in a MIN-CUT SET, it is ok

ii

if uv is in a MIN-CUT SET but not in another MIN-CUT SET, it is ok



can we safely contract this edge?

iii

if uv is in every MIN-CUT SET it is BAD

★ Edge uv is **BAD** if $uv \in E(C)$ for every MIN-CUT C

In other words, uv belongs to **all** MIN CUT SETS!

Let $K = |E(C)|$ be the size of the MIN CUT SETS

prop There are at most K BAD edges in G

CRUCIAL PROPERTY: there could be graphs in which each edge belongs to a MIN CUT SET. Still, only $\leq K$ edges are BAD

as $\leq k$ BAD edges

$$\Pr[uv \text{ is BAD}] \leq \frac{k}{m} \leq \frac{2}{nk} \cdot k = \frac{2}{n}$$

$$\sum_{u \in V} \deg(u) = 2m \quad + \quad \forall u \in V: \deg(u) \geq k$$

↑
handshaking
lemma

2

$$2m \geq nk \iff m \geq \frac{nk}{2}$$

Note This argument holds for multigraphs as $\deg(u)$ takes into account their multiplicities.

$P(n)$ = probability that GUESS MIN CUT correctly finds a MIN CUT in a graph with n nodes

$$P(n) \geq \begin{cases} 1 & n=2 \\ \left(1 - \frac{2}{n}\right) \cdot P(n-1) \end{cases}$$

the chosen
edge is NOT BAD
uv

conditional probability
given the choice uv

$$\Pr[A \cap B] = \Pr[A] \cdot \Pr[B|A]$$

Expanding it recursively...

$$P(n) \geq \left(\frac{n-2}{n}\right) \cdot \left(\frac{n-3}{n-1}\right) \cdot \left(\frac{n-4}{n-2}\right) \cdots \left(\frac{1}{3}\right) \cdot \underset{n=2}{1} = \frac{(n-2)!}{n! / 2} = \frac{2}{n(n-1)}$$

Do we like this? Not yet, we can boost it!

Repeat N times the execution of GUESS MIN CUT
and return the minimum value among those of the N executions

This idea works as GUESS MIN CUT returns a value $> k$
when a BAD edge is selected $\Rightarrow$ it's enough that one execution
is correct

Now the error probability is $\leq \underbrace{\left(1 - \frac{2}{n(n-1)}\right)^N}_{\text{error of a single execution}}$ $\Leftarrow$ we must be wrong all the N executions

$$\left(1 - \frac{2}{n(n-1)}\right)^N \sim e^{-\frac{2N}{n(n-1)}} = \frac{1}{n^c} \quad \text{w.h.p. for any constant } c > 1$$

$\uparrow$
 $(1-x) \sim e^{-x}$

$$\Rightarrow N = \Theta\left(c \frac{n(n-1)}{2} \cdot \ln n\right) = \Theta(n^2 \lg n)$$

► N can be further reduced

► In OR class, it can be shown how to find MIN CUT using FLOW algorithms

